Phase 2 — Transform

Created	@February 27, 2026 1:36 PM
Tags	

วัตถุประสงค์ของ Phase 2

ในเฟสนี้เราจะแปลงข้อมูลดิบจาก SQL Server ที่ Extract มาแล้วให้อยู่ในรูปแบบที่ถูกต้อง สะอาด และพร้อมสำหรับการ Load ลง MySQL ซึ่งในเฟสนี้มีการ:

1. Hash ข้อมูลส่วนตัว
2. แปลง Data Type ใน คอลัมน์ trans_date_trans_time
3. กรองข้อมูลที่ amt <= 0 หรือ NULL ออก
4. Rename columns

Phase นี้มี 5 Node (Node 4-8)ทำตามลำดับเลย

Node 4 — Java Snippet (SHA-256 Hash)

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Java Snippet` ใน Search Node
2. ลากมาวางต่อจาก Node 3

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 3: Java Snippet calculate the extraction time] ⟶ [Node 4: Java Snippet SHA-256]
```

ขั้นตอนที่ 3 : ตั้งค่า Output Columns

ส่วนที่ 1 : เพิ่ม Output Column

กดปุ่ม Add ในส่วน Output 4 ครั้ง สำหรับ column ที่ต้อง Hash:

Field Type	Replace	Name (Dropdown ข้าง Replace)	KNIME Type	Java Type	Java Field
Column	✅	`cc_num`	String	String	out_cc_num
Column	✅	`first`	String	String	out_first
Column	✅	`last`	String	String	out_last
Column	✅	`dob`	String	String	out_dob

> **สังเกต:** ครั้งนี้เลือก **Replace** ไม่ใช่ Append เพราะต้องการทับค่าเดิมด้วยค่า Hash

ส่วนที่ 2 : เพิ่ม Input Column

กดปุ่ม Add ในส่วน Input 4 ครั้ง:

Name	Java Type	Java Field
`cc_num`	String	`in_cc_num`
`first`	String	`in_first`
`last`	String	`in_last`
`dob`	String	`in_dob`

ขั้นตอนที่ 5 : ใส่ Code ใน Snippet

บรรทัด 13 (Your custom imports):

```
import java.security.MessageDigest;
```

บรรทัด 37 (Your custom variables):

```
String sha256(String input) throws Exception {
    if (input == null) return null;
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] hash = md.digest(input.getBytes("UTF-8"));
    StringBuilder sb = new StringBuilder();
    for (byte b : hash) {
        sb.append(String.format("%02x", b));
    }
    return sb.toString();
}
```

บรรทัด 42-45 (แทนที่ที่มีอยู่):

```
try{
    out_cc_num = sha256(in_cc_num);
    out_first = sha256(in_first);
    out_last = sha256(in_last);
    out_dob = sha256(in_dob);
} catch (Exception e) {
    throw new Abort(e.getMessage());
}
```

ขั้นตอนที่ 6 : Execute

คลิกขวา → **Execute** ถ้าสำเร็จแล้วเช็คว่า column `cc_num`, `first`, `last`, `dob` เปลี่ยนเป็น Hash แล้ว

Node 5 : String to DateTime

Node นี้แปลง `trans_date_trans_time` จาก String เป็น DateTime จริงๆตอน Extract โปรแกรม Knime ได้แปลง datatype จากString เป็น DateTime ให้แล้วอัตโนมัต แต่เพื่อให้มั่นใจว่า Format ถูกต้องตามที่ต้องการ และ ให้ขั้นตอนการ Transform มีความชัดเจนและสามารถตรวจสอบได้ในทุก Step จึงยังคงใส่ Node นี้ไว้

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `String to Date&Time` ใน Search Node
2. ลากมาวางต่อจาก Node 4

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 4: Java Snippet SHA-256] ──► [Node 5: String to Date&Time]
```

ขั้นตอนที่ 3 : ตั้งค่า

ตั้งค่า String to Date&Time ฝั่งขวามือ:

ส่วนที่ 1 : เอา column ที่ไม่ต้องการออก

ตอนนี้ Includes มีทุก column อยู่ กดปุ่ม **<<** เพื่อย้ายทุก column ไปฝั่ง Excludes ก่อน

ส่วนที่ 2 : เลือกเฉพาะ trans_date_trans_time

คลิกที่ `trans_date_trans_time` ในฝั่ง Excludes แล้วกด **>** เพื่อย้ายมาฝั่ง Includes

ส่วนที่ 3 : เลื่อนลงไปดู Input format

1. คลิกที่ **Date & time**
2. **Input format** จะเปลี่ยนเป็น:

   ```
   yyyy-MM-dd'T'HH:mm[:ss[.SSS]]
   ```

   > กดปุ่ม **Auto-guess format** แล้ว KNIME จะหา format ที่เหมาะสมให้อัตโนมัติ

3. **Output columns** เลือก **Replace** ได้เลย เพราะเราต้องการแทนที่ `trans_date_trans_time` ด้วย DateTime

---

ขั้นตอนที่ 4 : Execute

คลิกขวา → **Execute** แล้วเช็ค column `trans_date_trans_time`

Node 6 : Rule-based Row Filter

Node นี้กรองทิ้ง Row ที่ `amt <= 0` หรือ `amt` เป็น NULL

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Rule-based Row Filter` ใน Search Node

2. ลากมาวางต่อจาก Node 5

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 5: String to Date&Time] ──► [Node 6: Rule-based Row Filter]
```

ขั้นตอนที่ 3 : ตั้งค่า

คลิกขวา Node 6 กด configure แล้วใส่ Rule ในช่อง:

```
$amt$ <= 0 => FALSE
MISSING $amt$ => FALSE
TRUE => TRUE
```

> **อธิบาย:**
>
> - `$amt$ <= 0 => FALSE` = ถ้า amt น้อยกว่าหรือเท่ากับ 0 → ลบทิ้ง
> - `MISSING $amt$ => FALSE` = ถ้า amt เป็น NULL → ลบทิ้ง
> - `TRUE => TRUE` = นอกนั้นเก็บไว้ทั้งหมด

ขั้นตอนที่ 4 : Execute

คลิกขวา → **Execute** แล้วเช็คจำนวน Row ที่เหลือ

Node 7 : Column Rename

Node นี้เราจะเปลี่ยนชื่อคอลัมน์ 5 คอลัมน์ ได้แก่:

ชื่อเดิม	ชื่อใหม่
`trans_date_trans_time`	`trans_datetime`
`cc_num`	`cc_num_hash`
`first`	`first_hash`
`last`	`last_hash`
`dob`	`dob_hash`

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Column Rename` ใน Search Node
2. ลากมาวางต่อจาก Node 6

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 6: Rule-based Row Filter] ──► [Node 7: Column Rename]
```

ขั้นตอนที่ 3 : ตั้งค่า

ดับเบิลคลิก แล้วเปลี่ยนชื่อดังนี้:

ชื่อเดิม	ชื่อใหม่
`trans_date_trans_time`	`trans_datetime`
`cc_num`	`cc_num_hash`
`first`	`first_hash`
`last`	`last_hash`
`dob`	`dob_hash`

ขั้นตอนที่ 4 : Execute

คลิกขวา → **Execute** แล้วเช็คคอลัมน์ว่าเปลี่ยนชื่อตามที่ตั้งไว้หรือไม่

Node 8 : Java Snippet (Timer Stop for transform phase)

Node นี้คำนวณเวลาที่ใช้ใน Transform Phase ทั้งหมด

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Java Snippet` ใน Search Node
2. ลากมาวางต่อจาก Node 7

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 7: Column Rename] ──► [Node 8: Java Snippet Timer Stop for transf
orm phase]
```

ขั้นตอนที่ 3 : ตั้งค่า Input Columns

ส่วนที่ 1 : เพิ่ม Input Column

คลิกขวาที่ configure กด Add ในส่วน Input 1 ครั้ง:

Name	Java Type	Java Field
`extract_start_time`	LocalDateTime	`in_extract_start_time`

ส่วนที่ 2 : เพิ่ม Output Columns

กด Add ในส่วน Output 1 ครั้ง:

Append	Output Type	Java Field
`transform_sec`	Number (Float)	`out_transform_sec`

ขั้นตอนที่ 5 : ใส่ Code

บรรทัด 13 (Your custom imports):

```
import java.time.ZoneId;
```

บรรทัด 34-45 (แทนที่ที่มีอยู่):

```
long now = System.currentTimeMillis();

// แปลง LocalDateTime → milliseconds ก่อน
long extract_start_ms = in_extract_start_time
    .atZone(java.time.ZoneId.systemDefault())
    .toInstant()
    .toEpochMilli();

out_transform_sec = (now - extract_start_ms) / 1000.0;
```

ขั้นตอนที่ 6 : Execute

คลิกขวา → **Execute** แล้วเช็คว่า column `transform_sec` มีค่าเป็นวินาที